

APPENDIX F – ASSESSMENT DOCUMENTS

1. Indicator Assessment Rubrics

The following tables lists the general assessment rubrics for the performance indicators for each student outcome. The rubrics were developed in such a way as to be applicable to all courses within the curriculum via interpretation by individual instructors for any measures they assess.

Outcome (1): *Analyze a complex computing problem and to apply principles of computing and other relevant disciplines to identify solutions.*

Indicator	Below expectations	Meets expectations	Exceeds expectations
(1.1) Student can correctly interpret a computational problem and define its parameters	Student is unable to interpret problem correctly Student does not provide correct output for a small example	Student identifies correct approaches of what is needed for problem Student provides correct output for example problem	Student is able to generalize solution Student can connect approach to a broader class of problems
(1.2) Student can analyze a computation problem in order to choose mathematical and algorithmic principles that can be applied to solve the problem	Does not use correct principles to characterize the solution Does not see where to go with the problem	Correctly characterizes the problem with minor analysis errors Characterizes a broader problem	Uses correct principles to precisely characterize the problem Correct analysis of the problem
(1.3) Student can define a solution to a computational problem	Provides solution that does not address problem requirements	Provides solution solution with correct structure but minor errors and missing details	Provides efficient solution with rationale / explanation
(1.4) Student can effectively collect and document system requirements	Requirements are incomplete or vague Requirements are not documented in an appropriate form (e.g., use cases)	Requirements are fairly complete and specific; some important requirements may be missed Requirements are documented in an appropriate form; some details may be missing	Requirements are complete and precise Requirements are documented fully, with attention given to special cases, errors, etc.

(1.5) Student can effectively analyze and model a problem domain	Problem domain analysis is incomplete or vague Relationships between problem domain concepts are not modeled accurately	Problem domain analysis is relatively complete; some concepts may be missing Important relationships between concepts are modeled; opportunities for generalization may be missed	Problem domain analysis is very complete Relationships between concepts are modeled accurately; opportunities for generalization are exploited
(1.6) Student can identify the relative efficiency of different algorithms using asymptotic notation	Student does not understand the concept of asymptotic notation Student cannot subdivide an algorithm in order to analyze the running time of the parts Student cannot combine analysis of separate components of an algorithm into an overall running time Student cannot account for the effects of repetition (through iteration or recursion) on algorithm running time Student cannot use asymptotic bounds to choose between algorithms	Student can subdivide an algorithm in order to analyze the asymptotic bounds of the parts Student can combine the analysis of the parts into an overall asymptotic bound; possibly with some minor errors Student can account for effects of repetition; possibly with some minor errors Student can use asymptotic bounds to make informed choices for algorithms and data structures	Student can rigorously demonstrate asymptotic bound Student can analyze the effect of constant factors Student demonstrates solid understanding of how asymptotic bounds are related to actual (empirical) running time Student can construct models of expected workload and access patterns, along with asymptotic bounds, to make informed choices for data structures and algorithms

Outcome (2): *Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline.*

Indicator	Below expectations	Meets expectations	Exceeds expectations
(2.1) Student can identify and evaluate appropriate technologies to be used in a system	Language and/or platform is inappropriate for system requirements No discussion of language/platform choices	Language and platform are appropriate for system requirements Some discussion of language/platform choices; motivation for choosing one may not be fully explained or justified	Choice of language and/or platform is well justified by research, experiments, etc.
(2.2) Student can effectively construct a design model of a system	Design model is incomplete or vague Relationships between system concepts are not modeled accurately Design principles (e.g., SRP, OCP) are violated gratuitously	Design model is complete enough to address current system requirements; some requirements may not be addressed completely Important relationships between concepts are modeled; opportunities for generalization may be missed Design principles are followed to a reasonable extent; violations are relatively minor	Design model is very complete Opportunities for generalization are exploited Design principles are followed consistently
(2.3) Student can effectively incorporate requirements outside the problem domain (e.g., a user interface) into the design model	Non-problem-domain requirements are not addressed Non-problem-domain concepts are not properly separated from problem domain concepts	Non-problem-domain requirements are addressed adequately; some details could be improved Non-problem-domain concepts are mostly separated from problem domain concepts; abstraction boundaries could be strengthened	Non-problem-domain requirements are addressed very well Architectural patterns such as MVC, a persistence layer, etc. are employed to separate non-problem-domain concepts from problem domain concepts
(2.4) Student can plan and implement a testing strategy to ensure that system meets its quality goals	Planned/implemented tests are not adequate to ensure quality goal Test coverage is below a reasonable minimum	Planned/implemented tests are adequate to ensure quality goal Test coverage is adequate	Planned/implemented tests are comprehensive Test coverage is high and indicates testing of unusual paths

(2.5) Student can collect and analyze runtime benchmark data to characterize the efficiency of an algorithm or data structure	Student is unable to form an appropriate hypothesis about program performance Student is unable to employ instrumentation or profiling to collect performance data	Student can create a hypothesis about program performance related to a single subsystem, loop, bottleneck, etc. Student can employ instrumentation or profiling to collect performance data related to a hypothesis Student can use collected data to suggest implementation changes to improve performance	Student understands the performance effects caused by instrumentation and profiling Student can collect and analyze data from multiple program components in order to quantify interactions between components
(2.6) Student can specify appropriate security concerns and requirements for a component or system	Student does not demonstrate awareness of fundamental security vulnerabilities appropriate to the application Student does not demonstrate awareness of basic defense techniques for application	Student demonstrates awareness of the most important security vulnerability types for the application Student demonstrates understanding of essential defense techniques for application	Student demonstrates knowledge of more complex security vulnerability types for the application Student demonstrates awareness of advanced defense techniques
(2.7) Student can evaluate a component or system to identify security characteristics and identify vulnerabilities	Student cannot identify security-critical application code Student fails to identify instances of basic security vulnerabilities	Student can identify security-critical application code in most situations Student can identify instances common vulnerabilities such as SQL injection and buffer overflows	Student can identify complex application data flows that could result in unexpected code being security-critical Student can identify subtle vulnerabilities

Outcome (3): *Communicate effectively in a variety of professional contexts.*

Indicator	Below expectations	Meets expectations	Exceeds expectations
(3.1) Student can create and present a clear and well-organized technical presentation	Presentation is poorly organized Spoken remarks are difficult to understand or incoherent No diagrams are used Pacing is too slow or too fast Concepts are not explained adequately	Presentation has a clear organization Spoken remarks are comprehensible Appropriate diagrams are presented Pacing is appropriate Concepts are explained adequately	Presentation is very well organized Spoken remarks are clear and insightful Diagrams are used very effectively Concepts are explained very clearly
(3.2) Student can effectively incorporate technical content into a presentation using appropriate visual, textual, and spoken content	Inadequate description of problem Inadequate discussion of design No schematic representation of design (e.g., UML) Inadequate discussion of implementation issues No demonstration, or poor demonstration	Problem described adequately Appropriate discussion of design Appropriate schematic of design (e.g., UML) Appropriate discussion of implementation issues Adequate demonstration	Very good description of problem Discussion of evolution of design Schematic of design is very clear Discussion of how implementation issues led to design changes Demonstration highlights interesting implementation issues
(3.3) Student can write a clear and well-organized technical report	Report is poorly organized Writing quality is poor (numerous spelling and grammar mistakes) Terms and concepts are not explained adequately Sources are not cited Diagrams are not used effectively	Report has a clear organization; topics flow reasonably well Writing quality is generally good Terms and concepts are explained adequately Sources are cited Some diagrams are used where appropriate	Report has excellent organization; topics flow very well Writing quality is excellent Terms and concepts are explained very well Diagrams are used very effectively

Outcome (4): *Recognize professional responsibilities and make informed judgments in computing practice based on legal and ethical principles.*

Indicator	Below expectations	Meets expectations	Exceeds expectations
(4.1) Student can analyze and explain the ethical issues surrounding a particular computing topic (for example, peer-to-peer file sharing)	No awareness of ethical issues surrounding topic Does not consider points of views of all relevant stakeholders, or is unjustifiably biased No attempt to apply ethical principles to analysis of topic	General awareness of ethical issues surrounding topic Considers multiple points of views; analysis may have some unnecessary bias Some attempt to apply ethical principles; resulting judgment may have some flaws in reasoning	Strong awareness of ethical issues surrounding topic Balanced consideration of multiple points of view Well-reasoned application of ethical principles to make judgment
(4.2) Student demonstrates recognition of his or her professional responsibilities as a member of the computing profession	Lack of awareness of how professional work can affect others (employers, customers, clients, etc.) Lack of understanding of important professional obligations No discussion of regulatory and legal considerations	General awareness of how professional work can affect others General understanding of professional obligations Some discussion of regulatory and legal considerations; feasibility and social impact may not be fully considered	Strong awareness of how professional work can affect others Strong understanding of professional obligations Strong discussion of regulatory and legal implications; well-reasoned analysis of feasibility and social impact

Outcome (5): *Function effectively as a member or leader of a team engaged in activities appropriate to the program's discipline.*

Indicator	Below expectations	Meets expectations	Exceeds expectations
(5.1) Student demonstrates an ability to participate in and implement processes for team communication and coordination	Student does not communicate with team members Student does not coordinate his or her activities with team members	Student communicates regularly with team members Student coordinates his or her activities with team members	Student implements a process or processes for team communication (e.g., sets up a wiki, shared task document, etc.) Student implements a process or processes for team coordination (e.g., story wall, source code repository, etc.)
(5.2) Student demonstrates an ability to work closely with other students to solve technical problems	Student does not collaborate with team members to solve technical problems	Student is involved in some substantial technical collaboration with team members Student takes subsidiary or peer role in collaborations with team members	Student is involved in frequent technical collaboration with team members Student mentors team members on technical topics

Outcome (6): *Apply computer science theory and software development fundamentals to produce computing-based solutions.* [CS]

Indicator	Below expectations	Meets expectations	Exceeds expectations
(6.1) Student is proficient in a current programming language	Student cannot demonstrate proficiency in writing programs of moderate complexity in a current programming language	Student demonstrates proficiency by writing programs of moderate complexity Student can successfully use core language features and most commonly used idioms	Student demonstrates expertise by writing complex programs Student is able to employ advanced language features and idioms
(6.2) Student can create user interfaces using current platforms	Student cannot create user interfaces of moderate complexity using current platforms Student cannot separate data and application logic from presentation	Student can implement user interfaces of moderate complexity using current platforms Student can achieve some separation of data and application logic from presentation	Student can implement complex user interfaces using current platforms Student employs appropriate architectural patterns to separate data and application logic from presentation
(6.3) Student can write programs that use concurrency	Student does not understand when concurrency is desirable or necessary Student cannot implement programs using concurrency Student cannot use synchronization mechanisms to guarantee correct execution and consistency of shared data	Student understands when concurrency is desirable or necessary Student can implement programs using basic concurrency patterns such as producer/consumer, fork/join, etc. Student understands how to use synchronization to protect shared data Student has basic understanding of performance issues such as lock granularity	Student can use advanced concurrency patterns and idioms Student can using profiling to identify and overcome performance bottlenecks
(6.4) Student can implement automated tests to satisfy the goals of a testing strategy	Student does not implement any automated tests Automated tests are incorrect or incomplete	Student implements basic automated tests such as unit tests Automated tests cover reasonable subset of tested API	Student implements more sophisticated automated tests such as subsystem tests, integration tests, or UI tests Automated tests consider important corner cases

(6.5) Student can use appropriate implementation techniques and practices to meet security requirements and/or mitigate discovered vulnerabilities	Student's code contains basic security flaws such as trusting potentially malicious input Student fails to correct common security vulnerabilities (such as replacing SQL injection with use of placeholders for dynamic values) Student's code architecture implements security via ad-hoc mechanisms rather than a systematic approach	Student's code takes basic precautions to avoid common security vulnerabilities Student can correct common security vulnerabilities Student's code architecture contains at least some basic mechanisms such as reusable functions to consolidate handling of security concerns	Students code is hardened against a variety of attacks Student can correct a range of security vulnerabilities including more subtle ones Students code architecture treats security as a first-class concern and uses appropriate mechanisms (such as request filters) to implement security policies in a general way
---	---	--	--

2. Sample Indicator Faculty Course Assessment Report (FCAR)

(1.6) Analyze a complex computing problem and to apply principles of computing and other relevant disciplines to identify solutions.
- Student can identify the relative efficiency of different algorithms using asymptotic notation

Course: CS 360, Analysis of Algorithms

Work used: Empirical Comparison Report

Description

Students had implemented five sorting algorithms, collected empirical data for various input sizes, and plotted the data with an approximate fit of the asymptotic efficiency to estimate the hidden constant values. The report asked students to combine the data for all five sorts to create plots and corresponding discussion comparing them under a variety of conditions, e.g. different data set sizes and element ranges.

Target goal: 70% of students meeting or exceeding expectations

Needs improvement: 6/15 (40.0%) :111111
Meets expectations: 6/15 (40.0%) :111111
Exceeds expectations: 3/15 (20.0%) :111

Results: The target of 70% meeting or exceeding expectations was not achieved.

Summary/Observations: Incorporating a standard report template along with more explicit instructions on both the organization and content of the report significantly improved the quality of student submissions for one of the sections (which had primarily CE students). The other section (which had a majority of CS students), despite having multiple checkpoints with ample opportunity to resubmit drafts had the bulk of the needs improvement reports despite more explicit instructions and comparison exercises on several homework problems. Recommend more emphasis/instruction on performing empirical comparisons through plotting multiple data sets on a single graph. CE students did much better as a result of other engineering coursework that employs data collection and analysis.

CE

Needs improvement: 0/10 (0%) :
Meets expectations: 4/10 (40.0%) :1111
Exceeds expectations: 6/10 (60.0%) :111111